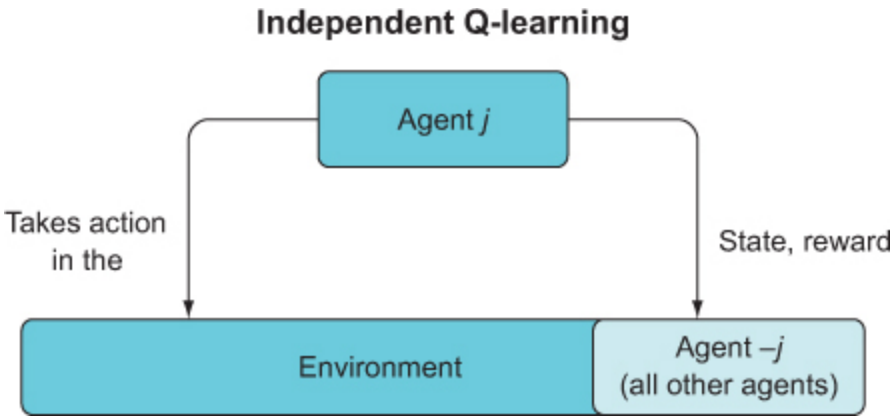
algorithm) for the various agents, and each agent sees the environment as it is and takes actions. If the agents we are trying to control all use the same policy, which is a reasonable assumption in some cases (e.g., in a multi-player game where each player is identical), then we could even re-use a single DQN (i.e., a single set of parameters) to model multiple agents.

This approach is called *independent Q-learning* (IL-Q), and it works reasonably well, but it misses the fact that interactions between agents affect the decision-making of each. With an IL-Q algorithm, each agent is completely unaware of what other agents are doing and how other agents' actions might affect itself. Each agent only gets a state representation of the environment, which includes the current state of each other agent, but it essentially treats the activity of other agents in the environment as noise since the behavior of other agents is, at most, only partially predictable (figure 9.2).

Figure 9.2. In independent Q-learning, an agent does not directly perceive the actions of other agents but rather pretends they are part of the environment. This is an approximation that loses the convergence guarantees that Q-learning has in the single-agent setting, since the other agents make the environment nonstationary.



In the ordinary Q-learning we've done so far, where there's only a single agent in the environment, we know the Q function will converge to the optimal value, so we will converge on an optimal policy (it is mathematically guaranteed to converge in the long run). This is because in the single-agent setting, the environment is *stationary*, meaning the distribution of rewards for a given action in a given state is always the same (figure 9.3). This stationary feature is violated in the multi-agent setting